

IPA ASSIGNMENT – 19

Name – Nishant Nahar

Roll No – 241551078

1. Implement color image sharpening using Laplacian filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_4.jpeg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
kernel = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], dtype=np.float32)
channels = cv2.split(img)
result = []
for channel in channels:
    laplacian = cv2.filter2D(channel, cv2.CV_32F, kernel)
    sharpened = channel.astype(np.float32) - laplacian
    sharpened = np.clip(sharpened, 0, 255).astype(np.uint8)
    result.append(sharpened)
sharpened = cv2.merge(result)
plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(sharpened)
plt.title("Sharpened Image")
plt.axis("off")
plt.show()
```

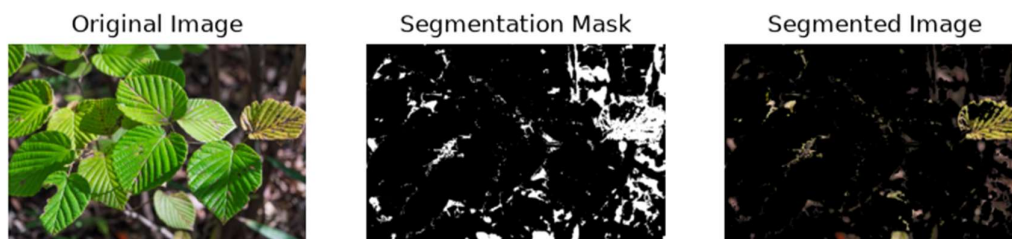
OUTPUT



2. Implement color segmentation on HSI model

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_4.jpeg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
hsi = cv2.cvtColor(img, cv2.COLOR_RGB2HSV)
lower = np.array([0, 50, 50])
upper = np.array([30, 255, 255])
mask = cv2.inRange(hsi, lower, upper)
segmented = cv2.bitwise_and(img, img, mask=mask)
plt.subplot(1, 3, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 3, 2)
plt.imshow(mask, cmap="gray")
plt.title("Segmentation Mask")
plt.axis("off")
plt.subplot(1, 3, 3)
plt.imshow(segmented)
plt.title("Segmented Image")
plt.axis("off")
plt.show()
```

OUTPUT



3. Implement color edge detection using di zenzo technique

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_4.jpeg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
r, g, b = cv2.split(img.astype(np.float32))
rx = cv2.Sobel(r, cv2.CV_32F, 1, 0, ksize=3)
ry = cv2.Sobel(r, cv2.CV_32F, 0, 1, ksize=3)
gx = cv2.Sobel(g, cv2.CV_32F, 1, 0, ksize=3)
```

```

gy = cv2.Sobel(g, cv2.CV_32F, 0, 1, ksize=3)
bx = cv2.Sobel(b, cv2.CV_32F, 1, 0, ksize=3)
by = cv2.Sobel(b, cv2.CV_32F, 0, 1, ksize=3)
J11 = rx**2 + gx**2 + bx**2
J22 = ry**2 + gy**2 + by**2
J12 = rx * ry + gx * gy + bx * by
theta = 0.5 * np.arctan2(2 * J12, J11 - J22)
G = np.sqrt(
    0.5 * (J11 + J22 + (J11 - J22) * np.cos(2 * theta) + 2 * J12 *
np.sin(2 * theta)))
G = cv2.normalize(G, None, 0, 255, cv2.NORM_MINMAX)
G = G.astype(np.uint8)
plt.subplot(1, 2, 1) plt.imshow(img)
plt.title("Original Image")
plt.axis("off") plt.subplot(1, 2, 2)
plt.imshow(G, cmap="gray")
plt.title("Di Zenzo Edge Detection")
plt.axis("off") plt.show()

```

OUTPUT



4. Implement noise elimination from a color image for Gaussian salt pepper and exponential noise

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread("img_4.jpeg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gaussian = img.astype(np.float32) + np.random.normal(0, 25, img.shape)
gaussian = np.clip(gaussian, 0, 255).astype(np.uint8)
salt_pepper = img.copy()
prob = 0.05
random = np.random.rand(*img.shape[:2])
salt_pepper[random < prob / 2] = 255

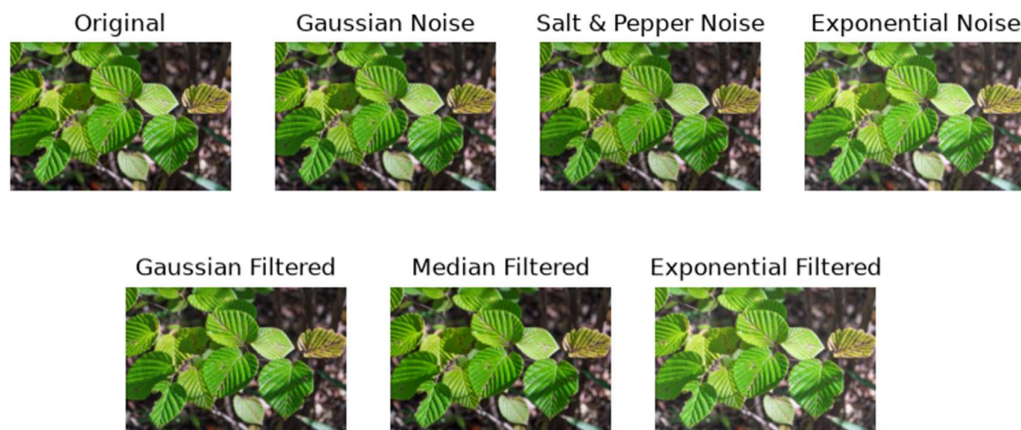
```

```

salt_pepper[random > 1 - prob / 2] = 0
exponential = img.astype(np.float32) + np.random.exponential(25,
img.shape)
exponential = np.clip(exponential, 0, 255).astype(np.uint8)
gaussian_filtered = cv2.GaussianBlur(gaussian, (5, 5), 0)
salt_pepper_filtered = cv2.medianBlur(salt_pepper, 5)
exponential_filtered = cv2.GaussianBlur(exponential, (5, 5), 0)
plt.figure(figsize=(12, 8))plt.subplot(2, 4, 1)
plt.imshow(img)plt.title("Original")
plt.axis("off")plt.subplot(2, 4, 2)
plt.imshow(gaussian) plt.title("Gaussian Noise")
plt.axis("off") plt.subplot(2, 4, 3)
plt.imshow(salt_pepper) plt.title("Salt & Pepper Noise")
plt.axis("off") plt.subplot(2, 4, 4)
plt.imshow(exponential) plt.title("Exponential Noise")
plt.axis("off") plt.subplot(2, 4, 6)
plt.imshow(gaussian_filtered) plt.title("Gaussian Filtered")
plt.axis("off") plt.subplot(2, 4, 7)
plt.imshow(salt_pepper_filtered) plt.title("Median Filtered")
plt.axis("off") plt.subplot(2, 4, 8)
plt.imshow(exponential_filtered) plt.title("Exponential Filtered")
plt.axis("off") plt.show()

```

OUTPUT



5. Implement image compression using Huffman coding

```

import cv2
import heapq
from collections import Counter
import matplotlib.pyplot as plt
img = cv2.imread("img_4.jpeg", 0)

```

```

freq = Counter(img.flatten())
heap = [[f, [s, ""]] for s, f in freq.items()]
heapq.heapify(heap)
while len(heap) > 1:
    lo = heapq.heappop(heap)    hi = heapq.heappop(heap)
    for pair in lo[1:]: pair[1] = "0" + pair[1]
    for pair in hi[1:]: pair[1] = "1" + pair[1]
    heapq.heappush(heap, [lo[0] + hi[0]] + lo[1:] + hi[1:])
codes = dict(heap[0][1:])
encoded = "".join(codes[pixel] for pixel in img.flatten())
original_bits = img.size * 8 compressed_bits = len(encoded)
compression_ratio = original_bits / compressed_bits
print("Original Size:", original_bits, "bits")
print("Compressed Size:", compressed_bits, "bits")
print("Compression Ratio:", compression_ratio)
plt.imshow(img, cmap="gray") plt.title("Original Image")
plt.axis("off") plt.show()

```

OUTPUT

Original Image



```

Original Size: 8875200 bits
Compressed Size: 8420814 bits
Compression Ratio: 1.0539598665877195

```